DATABASE MANAGEMENT SYSTEM (CT 652)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 4: Relational Database Design

Syllabus Hours: 8 hrs | Marks Weight: 16 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070–2082)

CHAPTER 4 SYLLABUS TOPICS

- **4.1** Pitfalls in Relational Database Design
- **4.2** Decomposition
- **4.3** Normalization using Functional Dependencies
- **4.4** First Normal Form (1NF)
- **4.5** Second Normal Form (2NF)
- **4.6** Third Normal Form (3NF)
- **4.7** Boyce-Codd Normal Form (BCNF)
- **4.8** Multivalued Dependencies and Fourth Normal Form (4NF)
- **4.9** Join Dependencies and Fifth Normal Form (5NF)

SECTION 1: Pitfalls in Relational Database Design & Decomposition

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q1. What are the pitfalls (anomalies) in relational database design? Explain insertion, deletion, and update anomalies with examples. Why do these occur?

[2+4+2]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Pitfalls in Relational Database Design

When a relational database is poorly designed — typically by combining multiple entities into a single relation — it suffers from data anomalies. These problems occur primarily because of redundancy (storing the same information in multiple places) and the mixing of concerns (combining attributes of different entities into one table).

Motivating Example — Bad Design (All-in-one table):

emp_id	emp_name	dept_id	dept_name	dept_location	salary
101	Ram	D1	CS	Kathmandu	50000
102	Sita	D1	CS	Kathmandu	55000
103	Hari	D2	IT	Pokhara	48000
104	Gita	D2	IT	Pokhara	52000

Anomaly Type	Definition	Example from Table Above	Root Cause
Insertion Anomaly	Cannot insert new data without inserting unrelated data. Or cannot insert an entity unless another related entity also exists.	Cannot insert a new department (D3, Finance, Lalitpur) unless we also insert an employee in it — but dept_id+emp_id are needed for every row.	Combining two entities (Employee + Department) in one table.
Deletion Anomaly	Deleting one entity accidentally destroys information about another entity.	If employee 103 (only employee in IT dept) is deleted, we lose ALL information about the IT department (D2, Pokhara) — it disappears from the database.	Information about two entities stored in the same row.
Update (Modification) Anomaly	Updating one fact requires updating it in many places. Inconsistency if not all copies are updated.	If the CS department moves from Kathmandu to Lalitpur, we must update dept_location in BOTH rows for emp 101 and 102. Missing one creates inconsistency.	Redundant storage of the same fact (dept_location) in multiple rows.

Most Repeated: "What are anomalies in DBMS? Explain with example." appears almost every year. Always give the bad table, then show all three anomalies — insertion, deletion, update — with specific examples from that table.

Q2. What is Decomposition? What are the desirable properties of a good decomposition? Explain Lossless Join and Dependency Preservation with examples.

[2+4+2]
2080 Ashwin 2078 Chaitra 2074 Bhadra

Answer: Decomposition

Decomposition is the process of splitting a poorly designed (unnormalized) relation R into two or more smaller relations R1, R2, ... Rn to eliminate redundancy and anomalies. The goal is that the decomposed relations together preserve all the information of the original relation while removing the pitfalls.

Desirable Properties of a Good Decomposition:

Property	Definition	Importance
Lossless Join Decomposition	When the decomposed relations R1 and R2 are joined (natural join), we get back exactly the original relation R — no extra (spurious) tuples and no missing tuples.	Ensures NO information is lost during decomposition. Critical — without this, queries on decomposed tables give wrong answers.
Dependency Preservation	Every functional dependency in the original relation F can be checked using only one of the decomposed relations, without needing to join them.	Ensures we can enforce all integrity constraints efficiently. Without it, checking FDs requires expensive joins.

No Redundancy	Each fact is stored exactly once across the decomposed relations.	Eliminates insertion, deletion, and update anomalies.

Lossless Join — Testing with FDs (Chase Algorithm):

A decomposition of R into R1 and R2 is LOSSLESS if and only if at least one of these holds:

- R1 ∩ R2 → R1 (i.e., the common attributes form a superkey for R1), OR
- R1 ∩ R2 → R2 (i.e., the common attributes form a superkey for R2)

Example:

R(A, B, C, D) with FDs: A → B, B → C, C → D. Decompose into R1(A,B,C) and R2(C,D).

- R1 ∩ R2 = {C}
- Does C → R1? C determines C but not A or B. NO.
- Does C → R2? C determines D (C → D is given). YES — C is superkey of R2(C,D). ✓
- Therefore, this decomposition is LOSSLESS. ✓

Dependency Preservation — Example:

R(A,B,C) with FDs: A → B, B → C. Decompose into R1(A,B) and R2(B,C).

- FD A → B: Can be checked on R1(A,B) alone. ✓ PRESERVED.
- FD B → C: Can be checked on R2(B,C) alone. ✓ PRESERVED.
- Both FDs preserved — this is a dependency-preserving decomposition. ✓

Exam Tip: BCNF always guarantees lossless join but may NOT preserve dependencies. 3NF always guarantees both lossless join AND dependency preservation. This is a key distinction to remember.

SECTION 2: Functional Dependencies & Armstrong's Axioms

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q3. What is a Functional Dependency (FD)? Explain trivial and non-trivial FDs. State and explain Armstrong's Axioms for inferring FDs. Compute the closure of an attribute set.

[1+2+4+1]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: Functional Dependencies

Functional Dependency (FD): A functional dependency X → Y on a relation schema R means that whenever two tuples in R agree on the values of attributes X, they must also agree on the values of attributes Y. Read as: "X functionally determines Y" or "Y is functionally dependent on X."

Example: In a Student relation, Student_ID → Name means if two tuples have the same Student_ID, they must have the same Name. Student_ID functionally determines Name.

Type	Definition	Example
Trivial FD	Y □ X — the right side is a subset of the left side. Always holds.	(A,B) → A or (A,B) → B or (A,B) → (A,B)
Non-Trivial FD	Y □ X — the right side contains at least one attribute not in the left side.	A → B (where B □ {A}) or (A,B) → C
Completely Non-Trivial FD	X ∩ Y = □ — the right side has NO overlap with the left side.	A → B where A ≠ B and B □ {A}

Armstrong's Axioms (Sound and Complete Inference Rules):

Axiom	Name	Rule	Example
A1	Reflexivity	If Y ⊆ X, then X → Y. (All trivial FDs)	(A,B) → A; (A,B,C) → (A,B)
A2	Augmentation	If X → Y, then XZ → YZ for any attribute set Z.	A → B ⟹ AC → BC
A3	Transitivity	If X → Y and Y → Z, then X → Z.	A→B and B→C □ A→C

These three axioms are SOUND (generate only valid FDs) and COMPLETE (can infer ALL valid FDs from a set F). Additional derived rules for convenience:

Derived Rule	Name	Rule	Example
A4	Union	If X→Y and X→Z, then X→YZ.	A→B and A→C □ A→BC
A5	Decomposition	If X→YZ, then X→Y and X→Z.	A→BC ⟹ A→B and A→C
A6	Pseudotransitivity	If X→Y and WY→Z, then WX→Z.	A→B and CB→D □ CA→D

Attribute Closure X+ (Computing Closure of a Set of Attributes):

The closure of an attribute set X under a set of FDs F, written X+, is the set of ALL attributes that can be functionally determined by X using Armstrong's Axioms.

Algorithm to Compute X+:

- Step 1: Initialize result = X (the given attribute set).
- Step 2: For each FD (A → B) in F: if A ⊆ result, then add B to result.
- Step 3: Repeat Step 2 until no more attributes can be added (fixed point).
- Step 4: result = X+

Example: R(A,B,C,D,E,F), FDs: A→BC, BC→DE, B→F. Compute A+.

A → BC
BC → DE
B → F

Step	result (X+)	Rule Applied
Start	{ A }	Initial: result = A
Apply A→BC	{ A, B, C }	A □ result, so add B, C
Apply BC→DE	{ A, B, C, D, E }	BC □ result, so add D, E
Apply B→F	{ A, B, C, D, E, F }	B □ result, so add F
No more	{ A, B, C, D, E, F }	Fixed point reached

A+ = {A,B,C,D,E,F} = all attributes of R → A is a CANDIDATE KEY of R (it determines everything). ✓

Q4. Define the following terms with examples: (i) Superkey (ii) Candidate Key (iii) Prime Attribute (iv) Non-prime Attribute (v) Canonical Cover (Minimal Cover). Also explain how to find a canonical cover.	**[3+4]** *2081 Chaitra 2078 Chaitra 2072 Ashwin*

Answer: Keys, Prime/Non-Prime Attributes and Canonical Cover

Term	Definition	Example for R(A,B,C,D), FDs: AB→CD, C→A, C→D
Superkey	A set of attributes that functionally determines ALL attributes of R. May have redundant attributes.	AB, ABC, ABD, ABCD are all superkeys.
Candidate Key	A MINIMAL superkey — removing any attribute from it makes it no longer a superkey.	AB is a candidate key (A alone → nothing useful; B alone → nothing useful). If C→A and C→D given, then C is also a candidate key since C→AD and C trivially→C.
Prime Attribute	An attribute that appears in AT LEAST ONE candidate key.	If candidate keys are AB and C: Prime attributes = A, B, C.
Non-Prime Attribute	An attribute that does NOT appear in any candidate key.	D does not appear in any candidate key → D is non-prime.

Canonical Cover (Fc): A canonical cover (minimal cover) of a set of FDs F is a simplified equivalent set Fc such that: (1) Fc is logically equivalent to F — same closure F+ = Fc+. (2) Every FD in Fc has no extraneous attributes on either side. (3) No FD in Fc can be removed without changing Fc+. (4) The left-hand side of each FD in Fc is unique (no two FDs with same LHS).

Algorithm to Find Canonical Cover:

- Step 1: Use the Union rule to combine FDs with the same LHS. e.g., A→B and A→C → A→BC.
- Step 2: Remove extraneous attributes from the LHS of each FD: For FD XA→Y, check if X→Y already follows — if yes, A is extraneous; remove it.
- Step 3: Remove extraneous attributes from the RHS of each FD: For FD X→YA, check if X→Y already follows without A — if yes, A is extraneous; remove it.
- Step 4: Remove redundant FDs: If removing an FD still leaves F+ unchanged, remove it.
- Step 5: Repeat until no further simplification is possible.

Example: R(A,B,C), FDs F = {A→BC, B→C, AB→C}. Find Fc.

- Step 1: A→BC already combined. No duplicate LHS.
- Step 2: Check AB→C — is B extraneous in LHS? Test A→C: from A→BC we get A→C. YES, B is extraneous. Remove: AB→C becomes A→C.
- Step 3: A→BC — is C extraneous in RHS? Test A→B without C: A→B is possible (A→BC by decomposition). Test A→C: still derivable from A→B and B→C. YES — but keep checking. Actually A→C is already an FD. So C in A→BC is extraneous. Simplify A→BC to A→B.
- Step 4: Now Fc = {A→B, B→C}. Check: is A→BC derivable? A→B, then B→C, so A→BC by transitivity. ✓

Canonical Cover: Fc = { A→B, B→C } (equivalent to original F but minimal)

SECTION 3: First and Second Normal Forms (1NF, 2NF)

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q5. What is normalization? Why is it needed? Define First Normal Form (1NF). How do you convert a non-1NF relation to 1NF? Give an example.

[1+2+2+3]
2081 Chaitra 2079 Chaitra 2076 Baisakh

Answer: Normalization and First Normal Form (1NF)

Normalization: Normalization is the process of organizing a relational database by decomposing tables to reduce data redundancy and eliminate data anomalies (insertion, deletion, update). It is done using a series of normal forms (1NF → 2NF → 3NF → BCNF → 4NF → 5NF), each progressively stricter.

Why Normalization is Needed:

- To eliminate data redundancy (same data stored multiple times).

- To prevent update, insertion, and deletion anomalies.
- To simplify database structure and improve data integrity.
- To make queries more efficient and maintainable.

First Normal Form (1NF): A relation is in 1NF if and only if: (1) All attribute values are ATOMIC (indivisible) — no multi-valued attributes, no repeating groups, no sets or lists in a cell. (2) Each column has a unique name. (3) All values in a column are of the same domain. (4) Each row is uniquely identifiable (has a primary key).

Violation of 1NF — Example:

emp_id	emp_name	phone_numbers	skills
101	Ram	9841000001, 9841000002	Java, Python, SQL
102	Sita	9851000001	C++, Java

This table violates 1NF because phone_numbers and skills contain multiple values (lists) in a single cell — they are NOT atomic.

Converting to 1NF:

Method 1: Create separate rows for each value (row explosion).

emp_id	emp_name	phone	skill
101	Ram	9841000001	Java
101	Ram	9841000001	Python
101	Ram	9841000001	SQL
101	Ram	9841000002	Java
102	Sita	9851000001	C++
102	Sita	9851000001	Java

Method 2 (preferred): Create separate tables for multi-valued attributes.

- Employee(emp_id, emp_name) — primary key: emp_id
- EmpPhone(emp_id, phone) — primary key: (emp_id, phone); FK: emp_id → Employee
- EmpSkill(emp_id, skill) — primary key: (emp_id, skill); FK: emp_id → Employee

Q6. Define Second Normal Form (2NF). What is a partial dependency? How do you convert a 1NF relation to 2NF? Give an example showing the conversion step by step.

[2+2+4]
2081 Ashwin 2079 Chaitra 2074 Bhadra

Answer: Second Normal Form (2NF)

Partial Dependency: A functional dependency X → Y is a PARTIAL dependency if X is a proper subset of some candidate key K (i.e., Y can be determined by PART of the candidate key rather than the WHOLE key). Partial dependencies only arise when the candidate key is composite (has more than one attribute).

Second Normal Form (2NF): A relation is in 2NF if and only if: (1) It is already in 1NF, AND (2) Every non-prime attribute is FULLY functionally dependent on EVERY candidate key — i.e., there are NO partial dependencies of non-prime attributes on any candidate key.

Example — Violation of 2NF:

Relation: OrderItem(order_id, product_id, quantity, product_name, customer_name)

Candidate Key: (order_id, product_id) — composite

order_id, product_id → quantity	*(Full FD — depends on whole key ✓)*
product_id → product_name	*(PARTIAL FD — depends only on product_id ✗)*
order_id → customer_name	*(PARTIAL FD — depends only on order_id ✗)*

Normal Form: 2NF Check	Violation: VIOLATION

Anomalies Due to Partial Dependency:

- Update Anomaly: Changing product_name requires updating all rows with that product_id.
- Insertion Anomaly: Cannot record a product's name without an order.
- Deletion Anomaly: Deleting the last order for a product loses the product name.

Converting to 2NF — Remove Partial Dependencies:

Decompose: For each partial dependency X → Y (where X is part of a candidate key), create a new table with X as the primary key and Y as its attribute.

- OrderItem_2NF(order_id, product_id, quantity) — PK: (order_id, product_id) — No partial deps ✓
- Product(product_id, product_name) — PK: product_id [extracted from partial dep product_id → product_name] ✓
- Order(order_id, customer_name) — PK: order_id [extracted from partial dep order_id → customer_name] ✓

All three tables are now in 2NF: every non-prime attribute is fully functionally dependent on the whole primary key.

SECTION 4: Third Normal Form (3NF) and BCNF

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q7. Define Third Normal Form (3NF). What is a transitive dependency? Convert a given relation to 3NF with a step-by-step example. What are the properties guaranteed by 3NF decomposition?	**[1+2+4+1]** *2081 Chaitra 2079 Chaitra 2076 Baisakh*

Answer: Third Normal Form (3NF)

Transitive Dependency: A functional dependency X → Z is a TRANSITIVE dependency in a relation if there exists an attribute set Y such that: X → Y and Y → Z, and Y is NOT a superkey, and Z is not a prime attribute. The dependency "transits" through Y — Z is not directly dependent on X but through an intermediary Y.

Third Normal Form (3NF): A relation is in 3NF if and only if: (1) It is in 2NF, AND (2) For every non-trivial FD X → Y in the relation, either: (a) X is a superkey of the relation, OR (b) Y is a prime attribute (part of some candidate key). In other words: NO non-prime attribute transitively depends on any candidate key.

Example — Violation of 3NF:

Relation: Employee(emp_id, dept_id, dept_name, dept_location)

Candidate Key: emp_id

emp_id → dept_id *(emp_id is PK — OK ✓)*
emp_id → dept_name *(TRANSITIVE via dept_id — ✗)*
emp_id → dept_location *(TRANSITIVE via dept_id — ✗)*
dept_id → dept_name, dept_location *(dept_id is NOT a superkey — VIOLATION ✗)*

Normal Form: 3NF Check	Violation: VIOLATION

The Problem (Anomalies):

- Update: Changing a department's location requires updating ALL employees in that department.
- Insertion: Cannot record a department's location without assigning it to at least one employee.
- Deletion: Deleting the last employee in a department loses the department's location.

Converting to 3NF — Remove Transitive Dependencies:

For each transitive dependency Y → Z (where Y is the intermediary and is not a superkey), create a new table with Y as the primary key.

- Employee_3NF(emp_id, dept_id) — PK: emp_id | FK: dept_id → Department ✓
- Department(dept_id, dept_name, dept_location) — PK: dept_id [extracted transitive dependency] ✓

Properties Guaranteed by 3NF Decomposition:

- Lossless Join: The decomposition always allows the original relation to be reconstructed via natural join. ✓
- Dependency Preservation: All FDs from the original relation can be checked on individual decomposed relations. ✓
- No Redundancy due to transitive deps: dept_location is now stored once per department. ✓

Q8. Define BCNF (Boyce-Codd Normal Form). How does it differ from 3NF? Convert the given relation to BCNF. What does BCNF guarantee that 3NF does not?

[2+4+2]
2080 Ashwin 2078
Chaitra 2073 Bhadra

Answer: Boyce-Codd Normal Form (BCNF)

BCNF (Boyce-Codd Normal Form): A relation R is in BCNF if and only if for every non-trivial functional dependency X → Y in R, X must be a SUPERKEY of R. BCNF is a stronger version of 3NF — it removes the exception in 3NF that allowed prime attributes to appear on the right side of FDs whose LHS is not a superkey.

Difference Between 3NF and BCNF:

Feature	3NF	BCNF
Condition for X→Y	X is a superkey, OR Y is a prime attribute.	X MUST be a superkey. No exceptions.
Strictness	Less strict — prime attribute exception allowed.	More strict — no exceptions at all.
Lossless Join	Always guaranteed. ✓	Always guaranteed. ✓
Dependency Preservation	Always guaranteed. ✓	NOT always guaranteed. May lose some FDs. ✗
Redundancy Removal	Removes most redundancy.	Removes ALL redundancy due to FDs.
Use When	Need dep. preservation + lossless join.	Maximum removal of redundancy; dep. preservation can be sacrificed.

Most Repeated: "Differentiate 3NF and BCNF. Convert the given relation to BCNF." appears in nearly every exam. Know: BCNF is stricter; removes all FD-based redundancy; may not preserve dependencies; 3NF always preserves dependencies.

Example — Relation in 3NF but NOT BCNF:

Relation: CourseAdviser(student_id, course_id, adviser_id)

Assumptions: Each course has multiple advisers. Each student is assigned one adviser per course. Each adviser advises only one course.

student_id, course_id → adviser_id	*(Candidate Key: (student_id, course_id))*
student_id, adviser_id → course_id	*(Candidate Key: (student_id, adviser_id))*
adviser_id → course_id	*(adviser_id → course_id: adviser_id is NOT a superkey! BCNF violation ✗)*

This relation IS in 3NF (course_id on RHS is a prime attribute — it's part of a candidate key). But NOT in BCNF (adviser_id → course_id and adviser_id is not a superkey).

Normal Form: BCNF Check	Violation: VIOLATION

Converting to BCNF:

For each FD X → Y that violates BCNF (X is not a superkey), decompose: Create R1(X ∪ Y) and R2(R − Y). Repeat until all relations are in BCNF.

- AdviserCourse(adviser_id, course_id) — PK: adviser_id [from adviser_id → course_id] ✓ BCNF
- StudentAdviser(student_id, adviser_id) — PK: (student_id, adviser_id) ✓ BCNF

Note: The FD student_id, course_id → adviser_id is LOST (cannot be checked on a single table). This is the dependency preservation trade-off of BCNF.

Q9. Normalize the following relation to 1NF, 2NF, 3NF, and BCNF step by step, clearly showing each step and the anomalies removed at each stage. R(A,B,C,D,E) with FDs: AB→C, C→D, D→E, AB→E.	**[2+2+2+2]** *2082 Kartik 2079 Chaitra 2077 Chaitra*

Answer: Step-by-Step Normalization to BCNF

Given: R(A,B,C,D,E), FDs: AB→C, C→D, D→E, AB→E

Step 1: Check for 1NF

Assumption: All attributes are atomic, single-valued, and unique. Table has a primary key.

R is already in 1NF. ✓

Step 2: Find Candidate Keys

- Compute (AB)+: AB→C (add C); C→D (add D); D→E (add E). (AB)+ = {A,B,C,D,E} = all attributes. ✓
- Can we remove A? B+ = {B} only. NOT all attributes. A cannot be removed.
- Can we remove B? A+ = {A} only. NOT all attributes. B cannot be removed.
- Therefore, AB is the ONLY candidate key. Prime attributes: A, B. Non-prime: C, D, E.

Step 3: Check for 2NF

Check for partial dependencies of non-prime attributes on the candidate key AB:

- AB→C: Does A→C? A+ = {A}. NO. Does B→C? B+ = {B}. NO. So AB→C is a FULL dependency. ✓
- AB→E: Does A→E? NO. Does B→E? NO. Full dependency. ✓ (also, AB→E is derivable via AB→C→D→E, so it is redundant — but not a partial dep)
- No partial dependencies exist. R is in 2NF. ✓

Step 4: Check for 3NF (Transitive Dependencies)

Check: Is there a non-prime attribute Y determined by a non-superkey non-prime intermediary?

- C→D: C is non-prime, D is non-prime, and C is NOT a superkey (C+ = {C,D,E} ≠ all). TRANSITIVE VIOLATION! ✗
- D→E: D is non-prime, E is non-prime, and D is NOT a superkey (D+ = {D,E} ≠ all). TRANSITIVE VIOLATION! ✗

Normal Form: 3NF	Violation: VIOLATION

Decompose to 3NF using 3NF Synthesis Algorithm:

- From canonical cover Fc = {AB→C, C→D, D→E}:
- R1(A, B, C) — PK: AB [from AB→C] ✓
- R2(C, D) — PK: C [from C→D] ✓
- R3(D, E) — PK: D [from D→E] ✓
- Check: AB is preserved in R1, so candidate key of R is preserved. All FDs are preserved. ✓

Step 5: Check BCNF for each decomposed relation

- R1(A,B,C) — FD: AB→C. LHS = AB = superkey of R1. ✓ IN BCNF.
- R2(C,D) — FD: C→D. LHS = C = superkey of R2 (C determines all attributes of R2). ✓ IN BCNF.
- R3(D,E) — FD: D→E. LHS = D = superkey of R3. ✓ IN BCNF.

Final Result: R1(A,B,C), R2(C,D), R3(D,E) — all in BCNF and 3NF.

Lossless join ✓

Dependency preserving ✓.

Q10. You are given the relation R(A,B,C,D) with the following functional dependencies: A→B, B→C, C→D, B→D. (i) Find all candidate keys. (ii) Find the canonical cover. (iii) Decompose into 3NF. (iv) Is the decomposition in BCNF? If not, convert to BCNF.	**[2+2+2+2]** *2081 Ashwin 2078 Baishakh 2075 Bhadra*

Answer: R(A,B,C,D) — Full Normalization Walkthrough

(i) Find all Candidate Keys:

- A+: A→B (add B); B→C (add C); C→D (add D). A+ = {A,B,C,D} = all. A is a superkey.
- Remove any subset? {}: empty set cannot determine A. So A is minimal → A is a CANDIDATE KEY.
- B+: B→C,D. B+ = {B,C,D} ≠ all (missing A). B is NOT a superkey.

- C+: C→D. C+ = {C,D} ≠ all. NOT a superkey.
- D+: D+ = {D} ≠ all. NOT a superkey.
- Only Candidate Key: A. Prime attribute: A. Non-prime: B, C, D.

(ii) Find Canonical Cover Fc:

A → B *(Keep)*
B → C *(Keep)*
C → D *(Keep)*
B → D *(B→D is REDUNDANT — derivable via B→C and C→D. REMOVE.)*

Canonical Cover: Fc = { A→B, B→C, C→D }

(iii) Decompose into 3NF (Synthesis Algorithm):

- For each FD in Fc, create a table: R1(A,B) PK:A, R2(B,C) PK:B, R3(C,D) PK:C.
- Check: Does any table contain the candidate key A? R1(A,B) contains A. ✓ (no need to add extra table)
- 3NF Decomposition: R1(A,B), R2(B,C), R3(C,D). Lossless ✓, Dependency-preserving ✓.

(iv) Check BCNF for each table:

- R1(A,B): A→B. A is PK and superkey. ✓ BCNF.
- R2(B,C): B→C. B is PK and superkey. ✓ BCNF.
- R3(C,D): C→D. C is PK and superkey. ✓ BCNF.

All tables are in BCNF. ✓ The decomposition is already in BCNF in this case.

SECTION 5: 4NF, 5NF, Multivalued & Join Dependencies

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin

Q11. What is a Multivalued Dependency (MVD)? How does it differ from a Functional Dependency? Define Fourth Normal Form (4NF). Convert a relation violating 4NF to 4NF with an example.	**[2+2+4]** *2081 Ashwin 2079 Chaitra 2075 Bhadra*

Answer: Multivalued Dependencies and 4NF

Multivalued Dependency (MVD): In a relation R(A, B, C), a multivalued dependency A □ B (read as "A multidetermines B") holds if: whenever two tuples have the same value of A, we can swap their B values and still get valid tuples. In other words, the set of B values associated with a given A value is independent of the C values.

Example — MVD in Practice:

Relation: CourseSkill(course, textbook, instructor)

Assumptions: A course can have MULTIPLE textbooks (independent of instructor), and MULTIPLE instructors (independent of textbook). There is no direct relationship between textbook and instructor.

course	textbook	instructor
DBMS	Korth	Ram
DBMS	Korth	Sita
DBMS	Navathe	Ram
DBMS	Navathe	Sita

course □ textbook (every combination of textbook and instructor must be present)

course □ instructor

This causes massive redundancy — every new textbook requires adding a row for every instructor.

Feature	Functional Dependency (FD)	Multivalued Dependency (MVD)
Notation	$X \rightarrow Y$	X □ Y
Meaning	One value of X determines exactly ONE value of Y.	One value of X is associated with a SET of Y values.
Trivial case	$X \rightarrow Y$ is trivial if $Y \subseteq X$.	$X \twoheadrightarrow Y$ is trivial if $Y \subseteq X$ or $X \cup Y$ = all attributes.
Requires	—	At least 3 attributes in the relation. Must have complementary MVD: X □ Y □ X □ Z (where Z = R−X−Y).
Cause of violation	FD violations cause 2NF/3NF/BCNF violations.	MVD violations cause 4NF violations.

Fourth Normal Form (4NF): A relation R is in 4NF if and only if for every non-trivial multivalued dependency X □ Y in R, X must be a SUPERKEY of R. (4NF implies BCNF.)

Converting CourseSkill to 4NF:

- Decompose each MVD into a separate table:
- CourseTextbook(course, textbook) — PK: (course, textbook) [from course $\twoheadrightarrow$ textbook] ✓
- CourseInstructor(course, instructor) — PK: (course, instructor) [from course $\twoheadrightarrow$ instructor] ✓

Now adding a new textbook for DBMS requires only ONE new row in CourseTextbook — not one per instructor. Redundancy eliminated. ✓

Q12. What is a Join Dependency? Define Fifth Normal Form (5NF). How does it differ from 4NF? Give an example of a relation in 4NF but not in 5NF.

[2+2+4]
2080 Ashwin 2078
Chaitra 2076 Baisakh

Answer: Join Dependencies and 5NF

Join Dependency (JD): A relation R satisfies a join dependency □[R1, R2, ..., Rn] if R can be losslessly decomposed into the projections R1, R2, ..., Rn — i.e., R = R1 □ R2 □ ... □ Rn. A join dependency with two components □[R1, R2] is equivalent to a multivalued dependency.

Fifth Normal Form (5NF / PJNF — Project-Join Normal Form): A relation R is in 5NF if and only if every non-trivial join dependency in R is implied by the candidate keys of R. A relation in 5NF cannot be decomposed further without losing information, unless the decomposition is based on candidate keys.

Example — Violation of 5NF (but in 4NF):

Relation: SupplierPartProject(supplier, part, project)

Business rule: Supplier S supplies Part P. Part P is used in Project J. Project J is supplied by Supplier S. In this case, the tuple (S,P,J) should exist.

supplier	part	project
S1	P1	J1
S1	P2	J2
S2	P1	J2
S1	P1	J2

This relation has a join dependency □[SP, PJ, SJ] where SP = (supplier,part), PJ = (part,project), SJ = (supplier,project). The relation can be losslessly decomposed into three binary projections — this is a 5NF violation if no single candidate key implies this JD.

5NF Decomposition:

- SP_Table(supplier, part) — PK: (supplier, part)
- PJ_Table(part, project) — PK: (part, project)
- SJ_Table(supplier, project) — PK: (supplier, project)

Joining these three tables reconstructs the original relation exactly (lossless). Redundancy from the cyclic constraint is eliminated.

Feature	BCNF	4NF	5NF
Based on	Functional Dependencies (FDs)	Multivalued Dependencies (MVDs)	Join Dependencies (JDs)
Condition	Every FD: LHS is superkey.	Every non-trivial MVD: LHS is superkey.	Every non-trivial JD implied by candidate keys.
Covers	FD redundancy	MVD redundancy	JD (cyclic) redundancy
Strictness	Less strict	Stricter than BCNF	Strictest
Practical Use	Most commonly used target.	Used when MVDs exist.	Rarely applied in practice.

SECTION 6: Complete Normalization Examples from Past Papers

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q13. Consider the relation R(A,B,C,D,E,F) with FDs: A→BC, CD→E, B→D, E→F. (i) Find all candidate keys. (ii) Find the canonical cover. (iii) Check if R is in 3NF or BCNF. (iv) Decompose R into 3NF.

[2+2+2+2]
2082 Kartik 2080 Ashwin

Answer: R(A,B,C,D,E,F) — Full Normalization

(i) Finding Candidate Keys:

- Compute A+: A→BC (add B,C); B→D (add D); CD→E (C,D in result: add E); E→F (add F). A+ = {A,B,C,D,E,F} = all. ✓ A is a superkey.
- Is A minimal? Removing A: {} → nothing. So A alone is the only single-attribute superkey.
- Candidate Key: A only. Prime attribute: A. Non-prime: B, C, D, E, F.

(ii) Canonical Cover:

- Step 1: Check for extraneous attributes. In CD→E: is C extraneous? Test D→E: D+ = {D} (no FD with D on LHS alone). NO. Is D extraneous? Test C→E: C+ = {C} (no FD). NO. Both needed.
- Step 2: Check for redundant FDs. No FD can be derived from the others without itself.

Canonical Cover: Fc = { A→BC, CD→E, B→D, E→F }

(iii) Check 3NF / BCNF:

- A→BC: A is candidate key (superkey). ✓ BCNF.
- CD→E: Is CD a superkey? CD+ = {C,D,E,F} ≠ all. NOT a superkey. E is non-prime. 3NF VIOLATION ✗ BCNF VIOLATION ✗

- B→D: Is B a superkey? B+ = {B,D} ≠ all. NOT a superkey. D is non-prime. 3NF VIOLATION ✗ BCNF VIOLATION ✗
- E→F: Is E a superkey? E+ = {E,F} ≠ all. NOT a superkey. F is non-prime. 3NF VIOLATION ✗ BCNF VIOLATION ✗

Normal Form: 3NF & BCNF	Violation: VIOLATION

(iv) Decompose into 3NF (Synthesis Algorithm):

- For each FD in Fc: R1(A,B,C) PK:A | R2(C,D,E) PK:CD | R3(B,D) PK:B | R4(E,F) PK:E
- Check: Does any table contain the candidate key A? R1 contains A. ✓
- 3NF Decomposition: R1(A,B,C), R2(C,D,E), R3(B,D), R4(E,F)

All tables are in 3NF. Lossless join ✓. Dependency preserving ✓.

Q14. Given the relation R(Emp_ID, Emp_Name, Dept_ID, Dept_Name, Project_ID, Project_Name, Hours). Assume: Each employee belongs to one department. Each department has a unique name. Each employee works on multiple projects. Normalize this relation completely to BCNF.	**[2+2+2+2]** *2081 Chaitra 2079 Chaitra 2077 Chaitra*

Answer: Employee/Department/Project — Full Normalization to BCNF

Step 0: Identify FDs from the problem description:

Emp_ID → Emp_Name, Dept_ID
Dept_ID → Dept_Name
Emp_ID, Project_ID → Hours
Project_ID → Project_Name

Step 1: Check 1NF

Assuming all attributes are atomic, a row is identifiable by (Emp_ID, Project_ID) — one employee can appear on multiple rows (one per project). 1NF: ✓

Step 2: Check 2NF — Remove Partial Dependencies

Candidate Key: (Emp_ID, Project_ID). Partial dependencies:

- Emp_ID → Emp_Name, Dept_ID (partial — depends only on Emp_ID, part of the key). REMOVE ✗
- Dept_ID → Dept_Name (transitive — not even direct from key). REMOVE ✗
- Project_ID → Project_Name (partial — depends only on Project_ID). REMOVE ✗
- (Emp_ID, Project_ID) → Hours (full — depends on complete key). KEEP ✓

2NF Decomposition:

- EmpProject(Emp_ID, Project_ID, Hours) — PK: (Emp_ID, Project_ID) ✓
- EmpDept(Emp_ID, Emp_Name, Dept_ID) — PK: Emp_ID ✓

- Project(Project_ID, Project_Name) — PK: Project_ID ✓
- (Dept_ID → Dept_Name still violates 3NF in EmpDept — handle next)

Step 3: Check 3NF — Remove Transitive Dependencies in EmpDept:

- In EmpDept(Emp_ID, Emp_Name, Dept_ID): Emp_ID → Dept_ID → Dept_Name. TRANSITIVE VIOLATION ✗

3NF Decomposition of EmpDept:

- Employee(Emp_ID, Emp_Name, Dept_ID) — PK: Emp_ID | FK: Dept_ID ✓
- Department(Dept_ID, Dept_Name) — PK: Dept_ID ✓

Final BCNF Schema (4 tables):

Table	Attributes	Primary Key	Foreign Keys
Employee	Emp_ID, Emp_Name, Dept_ID	Emp_ID	Dept_ID → Department
Department	Dept_ID, Dept_Name	Dept_ID	—
Project	Project_ID, Project_Name	Project_ID	—
EmpProject	Emp_ID, Project_ID, Hours	(Emp_ID, Project_ID)	Emp_ID → Employee; Project_ID → Project

All four tables are in BCNF. Lossless join ✓. Dependency preserving ✓. No anomalies remain. ✓

Q15. Consider R(ABCDE) with FDs: AB→C, D→E, B→D. Find all candidate keys. Is R in 2NF, 3NF, BCNF? Decompose R into BCNF.	**[2+2+2+2]** *2079 Ashwin Back 2075 Bhadra 2072 Ashwin*

Answer: R(A,B,C,D,E) — Candidate Keys + Normalization

(i) Finding Candidate Keys:

- Compute (AB)+: AB→C (add C); B→D (add D); D→E (add E). (AB)+ = {A,B,C,D,E} = all. AB is a superkey.
- Minimal? A alone: A+ = {A}. B alone: B+ = {B,D,E}. Neither alone determines all.
- So AB is a CANDIDATE KEY. Prime attributes: A, B. Non-prime: C, D, E.

(ii) Check 2NF (Partial Dependencies):

- B→D: B is PART of the candidate key AB. D is non-prime. PARTIAL DEPENDENCY ✗ — 2NF VIOLATION!

Normal Form: 2NF Check	Violation: VIOLATION

(iii) Check 3NF (Transitive Dependencies):

- B→D and D→E: B is part-of-key but not itself a key. D→E where D is non-prime and not a superkey. TRANSITIVE ✗
- AB→C: AB is superkey. ✓

Normal Form: 3NF Check	Violation: VIOLATION

(iv) Check BCNF:

- B→D: B is not a superkey. VIOLATION ✗
- D→E: D is not a superkey. VIOLATION ✗

Normal Form: BCNF Check	Violation: VIOLATION

(v) Decompose to BCNF:

- B→D violates BCNF. Decompose: R1(B,D) and R2(A,B,C,E) (R minus D, keeping B as join attribute).
- Check R2(A,B,C,E) with FDs: AB→C, and project D→E → becomes E is still in R2. Check: B→E (by transitivity B→D→E). So B→E holds on R2. B is not superkey of R2(A,B,C,E). STILL VIOLATES BCNF.
- Further decompose R2: B→E violates BCNF. Decompose: R3(B,E) and R4(A,B,C).
- Check R3(B,E): B→E. B is superkey (B determines all of R3). ✓ BCNF.
- Check R4(A,B,C): AB→C. AB is superkey of R4. ✓ BCNF.
- Check R1(B,D): B→D. B is superkey of R1. ✓ BCNF.

Final BCNF Decomposition: R1(B,D), R3(B,E), R4(A,B,C). Lossless join ✓.

Exam Tip: When decomposing to BCNF, always re-project the remaining FDs onto each sub-relation and check again. BCNF decomposition may need multiple steps.

SECTION 7: 3NF Synthesis Algorithm and Additional Practice

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2079 Chaitra, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin, 2070 Magh

Q16. Explain the 3NF Synthesis Algorithm. Apply it to decompose R(A,B,C,D,E) with FDs: A→B, BC→D, B→E into 3NF. Prove that the result is lossless and dependency-preserving.	**[3+4+1]** *2081 Chaitra 2078 Chaitra 2075 Bhadra*

Answer: 3NF Synthesis Algorithm

The 3NF Synthesis Algorithm (Steps):

- Step 1: Compute the Canonical Cover Fc of the given set of FDs F.
- Step 2: For each FD X → Y in Fc, create a relation schema Ri(X ∪ Y) with X as the primary key.
- Step 3: If none of the created schemas contains a candidate key of the original relation R, add one more schema containing only the candidate key attributes.

- Step 4: Remove any schema Ri that is a subset of another schema Rj (redundant schemas).

Application: R(A,B,C,D,E), FDs: A→B, BC→D, B→E

Step 1: Compute Canonical Cover Fc:

- Check A→B: Is it redundant? Remove and see if A→B still derivable: A+ without A→B = {A}. NOT derivable. KEEP.
- Check BC→D: Is C extraneous in LHS? B→D? B+ = {B,E}. D ∉ B+. NO. Is B extraneous? C→D? C+ = {C}. NO. Both needed.
- Check B→E: redundant? Without it: B+ = {B}. E not derivable. KEEP.

Fc = { A→B, BC→D, B→E }

Step 2: Create Relation Schemas from Fc:

- R1(A,B) — PK: A [from A→B]
- R2(B,C,D) — PK: BC [from BC→D]
- R3(B,E) — PK: B [from B→E]

Step 3: Check for Candidate Key

- Find candidate key of R: A+ = {A,B,E} (A→B, B→E). Not all. BC: BC→D, B→E. (BC)+ = {B,C,D,E}. Not all. AC: A→B, BC→D (now have B and C via A and C input): AC→B→E and BC→D. (AC)+ = {A,B,C,D,E} = all. ✓
- Candidate Key: AC. Does any Ri contain AC? R1 has A not C. R2 has BC not A. R3 has B not A or C. NONE contains AC.
- → Add R4(A,C) — PK: (A,C) [to preserve candidate key]

Step 4: Remove Redundant Schemas (if any):

- R1(A,B) — not subset of R2,R3,R4. KEEP.
- R2(B,C,D) — not a subset of any other. KEEP.
- R3(B,E) — not a subset of any other. KEEP.
- R4(A,C) — not a subset of any other. KEEP.

Final 3NF Decomposition: R1(A,B), R2(B,C,D), R3(B,E), R4(A,C)

Proving Lossless Join:

- R1 ∩ R4 = {A}. A → B (A is PK of R1). A is superkey of R1. ✓ Lossless for R1 ⋈ R4.
- R4 ∩ R2 = {C}. C alone does not determine all of R2. But we can chain: R1 □ R4 gives (A,B,C). Then (A,B,C) ⋈ R2 on BC → full R2 accessible. Lossless chain. ✓

Proving Dependency Preservation:

- A→B: Checkable on R1(A,B). ✓
- BC→D: Checkable on R2(B,C,D). ✓
- B→E: Checkable on R3(B,E). ✓

All FDs in Fc are preserved. ✓

Q17. What is the difference between 3NF and BCNF? When would you prefer 3NF over BCNF? Given FDs, show a relation that is in 3NF but not BCNF.

[2+2+4]
2080 Ashwin 2078 Baishakh 2073 Bhadra

Answer: 3NF vs BCNF — Detailed Comparison

Aspect	3NF	BCNF
Definition	For every non-trivial FD X→Y: X is superkey OR Y is prime attribute.	For every non-trivial FD X→Y: X MUST be a superkey. No exceptions.
Strictness	Less strict (prime attribute exception).	More strict — no exceptions at all.
Lossless Join	Always guaranteed by synthesis algorithm. ✓	Always guaranteed. ✓
Dependency Preservation	Always guaranteed. ✓	NOT always guaranteed. ✗
Redundancy Removal	Removes most redundancy. Some residual redundancy may remain.	Removes ALL redundancy caused by FDs.
When to Prefer	When dependency preservation is critical (e.g., must enforce all FDs without joins).	When removing all FD redundancy is paramount; can sacrifice dep. preservation.
Decomposition Algorithm	3NF Synthesis — guaranteed to give lossless, dep-preserving result.	BCNF Decomposition — may need to sacrifice dependencies.

A Relation in 3NF but NOT in BCNF (Classic Example):

R(J, K, L) — represents a course-teacher-student booking system.

Business rules: (1) Each student K is assigned exactly one teacher J for each course L. (2) Each teacher J teaches only one course L.

J, K → L	*(Candidate Key (J,K) — student K assigned teacher J for course L)*
K, L → J	*(Candidate Key (K,L) — for student K in course L, exactly one teacher J)*
J → L	*(Each teacher J teaches only one course — J NOT a superkey! BCNF VIOLATION ✗)*

3NF Check: J→L — L is a prime attribute (part of candidate key KL). So 3NF exception allows this. R is in 3NF. ✓

BCNF Check: J→L — J is NOT a superkey (J+={J,L} ≠ all). BCNF VIOLATION. ✗

If we decompose to BCNF: R1(J,L) and R2(J,K). Then FD KL→J is LOST — cannot be checked on either R1 or R2 alone. This is why 3NF is preferred here.

QUICK REVISION: Chapter 4 — Key Points Summary		
Topic	**Key Points**	**Expected Marks**
Anomalies	Insertion (can't add without related entity), Deletion (deleting one entity loses another), Update (must change multiple places). Caused by mixing entities in one table.	2-4
Decomposition	Good decomposition: Lossless Join + Dependency Preservation. Test lossless: R1∩R2→R1 or R1∩R2→R2.	2-4
Functional Dependency	X→Y: same X values → same Y values. Trivial: Y⊆X. Armstrong's Axioms: Reflexivity, Augmentation, Transitivity (+ Union, Decomposition, Pseudotransitivity).	2-4
Attribute Closure X+	Start with X; repeatedly apply FDs to grow the set; fixed point = X+. Use to find candidate keys, test FD derivability.	2-4
Canonical Cover Fc	Minimal equivalent FD set: no extraneous LHS/RHS attributes, no redundant FDs, unique LHS. Algorithm: extraneous attr removal + redundant FD removal.	2-4
1NF	All values atomic (no multi-valued, no repeating groups). Solution: separate tables for multi-valued attributes.	2-4
2NF	1NF + no partial dependency of non-prime attribute on any candidate key. Only an issue with composite keys. Fix: extract partial deps into separate tables.	3-6
3NF	2NF + no transitive dependency of non-prime attribute via non-superkey. Fix: extract transitive deps. 3NF synthesis: always lossless + dep-preserving. ✓✓	4-8
BCNF	For every non-trivial X→Y: X is superkey. Stricter than 3NF. Always lossless ✓ but may NOT preserve dependencies ✗. Preferred when redundancy removal > dep preservation.	4-8
3NF vs BCNF	3NF: lossless+dep-preserving but may have residual redundancy. BCNF: no redundancy but may lose dep preservation. Classic example: JKL relation.	2-4
MVD & 4NF	X□Y: set of Y-values independent of rest. 4NF: every non-trivial MVD X□Y has X as superkey. Fix: decompose each MVD into its own table.	2-4

JD & 5NF	Join dependency □[R1,...,Rn]: R=R1□...□Rn losslessly. 5NF: every non-trivial JD implied by candidate keys. Practical only for 3-way cyclic relationships.	2-4